

The Coward King

Entity-Relationship Diagram — Relational Schema Justification

Silent Crown · Tec de Monterrey

José Abel Domínguez

Leonardo Flores

Nicolas Casillas

This document presents the initial Entity-Relationship schema proposed for The Coward King database. Ten main entities are identified, along with their attributes with implicit domain, and the relationships between them with their cardinalities and referential integrity constraints. The schema has not been normalized to a specific normal form, as this is an initial version subject to change.

1. Entities and Attributes

PLAYER

Represents each registered user in the game. Stores credentials and creation date. It is the root entity of the model: every run, upgrade, and statistic belongs to a player. Enables the login/signup system described in the Main Menu screen.

Attribute	Domain / Type	Constraint	Description
player_id	INT	PK / NOT NULL	Unique player identifier
username	VARCHAR(50)	NOT NULL / UNIQUE	Display name shown on screen
created_at	TIMESTAMP	NOT NULL	Player registration date

RUN

Records each attempt (run) by a player. Captures the current level and horde, accumulated gold, and the final result (victory, defeat, in progress). Essential for the level checkpoint system and the Game Over screen, which needs to know how much gold is lost and which upgrades are retained.

Attribute	Domain / Type	Constraint	Description
run_id	INT	PK / NOT NULL	Unique run identifier
player_id	INT	FK → PLAYER	Owner player
current_level_id	INT	FK → LEVEL	Current level of the run
total_gold_earned	INT	DEFAULT 0	Total gold earned in the run
current_horde	INT	NOT NULL	Current horde of the run

Attribute	Domain / Type	Constraint	Description
started_at	TIMESTAMP	NOT NULL	Run start time
ended_at	TIMESTAMP	NULLABLE	Run end time
result	VARCHAR(20)	NOT NULL	Run result
is_active	BOOLEAN	DEFAULT TRUE	Indicates if the run is ongoing
speed	INT	NOT NULL	Speed or pace associated with the run

LEVEL

Static catalog table. Defines the three game levels (Catacombs, Dungeon, Castle) with their boss, visual theme, and number of hordes. Since this is fixed game design data, centralizing it avoids hardcoding it in the application logic and facilitates future balancing.

Attribute	Domain / Type	Constraint	Description
level_id	INT	PK / NOT NULL	Unique level identifier
level_number	INT	NOT NULL	Order of the level in the game
name	VARCHAR(50)	NOT NULL	Level name
theme	VARCHAR(100)	NOT NULL	Visual theme or environment
total_hordes	INT	NOT NULL	Total number of hordes in the level
boss_name	VARCHAR(50)	NOT NULL	Name of the level's final boss

HORDE

Defines the hordes within each level. Records balance parameters such as maximum turns and enemies per spawn. By separating HORDE from LEVEL, each horde can have independent configuration, which is useful if difficulty per horde within the same level needs to vary in the future.

Attribute	Domain / Type	Constraint	Description
horde_id	INT	PK / NOT NULL	Unique horde identifier
level_id	INT	FK → LEVEL	Level the horde belongs to
horde_number	INT	NOT NULL	Horde number within the level
max_turns	INT	NOT NULL	Maximum allowed turns
enemies_per_spawn	INT	NOT NULL	Enemies generated per turn

RUN_HORDE

Association table between a run and the hordes the player has played. Records concrete progress: turns survived and enemies eliminated per horde. It is the data source for the statistics screen (Enemies Defeated, etc.) and for validating the checkpoint upon death.

Attribute	Domain / Type	Constraint	Description
run_horde_id	INT	PK / NOT NULL	Unique relationship identifier
run_id	INT	FK → RUN	Run this entry belongs to
horde_id	INT	FK → HORDE	Horde played
turns_survived	INT	NOT NULL	Number of turns survived
enemies_killed	INT	NOT NULL	Enemies eliminated in the horde
completed	BOOLEAN	DEFAULT FALSE	Indicates if the horde was completed

CARD

Static catalog of the 15 cards in the game. Stores the base stats of each card (AP cost, HP, damage) and its type (attack, defense, political). This table is the reference point for the upgrade system: upgrades are deltas over the base values in CARD.

Attribute	Domain / Type	Constraint	Description
card_id	INT	PK / NOT NULL	Unique card identifier
name	VARCHAR(50)	NOT NULL / UNIQUE	Card name
type	VARCHAR(30)	NOT NULL	Card type
base_ap_cost	INT	NOT NULL	Base action point cost
base_hp	INT	NOT NULL	Base hit points
base_damage	INT	NOT NULL	Base damage
effect_description	VARCHAR(200)	NOT NULL	Effect description

CARD_UPGRADE

Records the permanent upgrades a player has purchased for each card. Upgrades survive deaths and accumulate across runs. Allows reconstructing the current stats of each card by adding the bonuses to the base value in CARD. Modeling this as a separate table (instead of columns in CARD) allows a complete history of gold spending per player.

Attribute	Domain / Type	Constraint	Description
upgrade_id	INT	PK / NOT NULL	Unique upgrade identifier
player_id	INT	FK → PLAYER	Owner player
card_id	INT	FK → CARD	Upgraded card
level	INT	NOT NULL	Current upgrade level of the card
ap_cost_bonus	INT	DEFAULT 0	AP cost modifier

Attribute	Domain / Type	Constraint	Description
hp_bonus	INT	DEFAULT 0	Hit point bonus
damage_bonus	INT	DEFAULT 0	Damage bonus
gold_spent	INT	NOT NULL	Total gold invested in the upgrade

RUN_HAND

Records which cards form the player's hand in each horde of a run. Allows reconstructing the hand of any horde for analysis. The is_retained field indicates whether that card was the one chosen to be kept during the horde transition (the 1-card retention mechanic).

Attribute	Domain / Type	Constraint	Description
hand_id	INT	PK / NOT NULL	Unique entry identifier
run_id	INT	FK → RUN	Run this hand belongs to
horde_id	INT	FK → HORDE	Horde where the hand was used
card_id	INT	FK → CARD	Card included in the hand
is_retained	BOOLEAN	DEFAULT FALSE	Indicates if the card was retained

ENEMY

Static catalog of enemy types by level (Skeleton, Ogre, Elite Warrior and their boss versions). Stores their base stats and the is_boss flag to differentiate the boss summon mechanic. Being linked to LEVEL allows knowing which enemies appear in each level without hardcoding the logic in the application.

Attribute	Domain / Type	Constraint	Description
enemy_id	INT	PK / NOT NULL	Unique enemy identifier
level_id	INT	FK → LEVEL	Level the enemy belongs to
name	VARCHAR(50)	NOT NULL	Enemy name
type	VARCHAR(30)	NOT NULL	Enemy type
hp	INT	NOT NULL	Enemy hit points
damage	INT	NOT NULL	Damage dealt by the enemy
is_boss	BOOLEAN	DEFAULT FALSE	Indicates if the enemy is a boss; differentiates the boss summon mechanic

STATS

Aggregates global player statistics across all their runs (not per individual run). Allows displaying the Stats screen in the main menu with data such as total attempted runs, completed levels, eliminated enemies, and gold earned. It is a single-row table per player (1:1 relationship with PLAYER).

Attribute	Domain / Type	Constraint	Description
stat_id	INT	PK / NOT NULL	Unique record identifier
player_id	INT	FK → PLAYER / UNIQUE	Owner player
total_runs	INT	DEFAULT 0	Total runs started
total_enemies_killed	INT	DEFAULT 0	Total enemies eliminated
total_upgrades	INT	DEFAULT 0	Total upgrades performed
total_gold_earned	INT	DEFAULT 0	Total gold earned historically
levels_completed	INT	DEFAULT 0	Total levels completed

2. Relationships, Cardinalities, and Constraints

The following table summarizes all relationships in the model with their cardinality in both directions and the applicable referential integrity constraints.

Relationship	Cardinality	Referential Integrity Constraint
PLAYER → RUN	1 player : N runs. A player can have many runs.	ON DELETE CASCADE: deleting a player removes all their runs.
PLAYER → CARD_UPGRADE	1 player : N upgrades. A player can upgrade multiple cards.	ON DELETE CASCADE: deleting a player removes all their upgrades.
PLAYER → STATS	1 player : 1 stats. Each player has one stats record.	ON DELETE CASCADE; player_id UNIQUE in STATS.
RUN → LEVEL	N runs : 1 level. Many runs can be associated with a current level.	ON DELETE RESTRICT: a level cannot be deleted if runs are associated.
RUN → RUN_HORDE	1 run : N horde records. Each run can record multiple hordes played.	ON DELETE CASCADE.
RUN → RUN_HAND	1 run : N hand entries. Each run can have multiple cards recorded per horde.	ON DELETE CASCADE.
LEVEL → HORDE	1 level : N hordes. Each level contains multiple hordes.	ON DELETE CASCADE; UNIQUE(level_id, horde_number).
LEVEL → ENEMY	1 level : N enemies. Each level can define multiple enemy types.	ON DELETE CASCADE.
HORDE → RUN_HORDE	1 horde : N records in runs. The same horde can be played in different runs.	ON DELETE RESTRICT.
HORDE → RUN_HAND	1 horde : N hand entries. A horde can have many cards recorded across different runs.	ON DELETE RESTRICT.
CARD → RUN_HAND	1 card : N hand entries. A card can appear in many hands across different runs and hordes.	ON DELETE RESTRICT.

Relationship	Cardinality	Referential Integrity Constraint
CARD → CARD_UPGRADE	1 card : N upgrades. A card can have upgrades from different players.	ON DELETE RESTRICT.

3. Additional Integrity Constraints

To maintain data consistency, the schema includes additional validation constraints beyond those already defined in the relational model:

Constraint Type	Applies To	Rule / Description
CHECK (non-negative)	total_gold_earned, gold_spent, turns_survived, enemies_killed, hp, damage, all stats	Numeric values must be ≥ 0 . Negative amounts are not valid in any game context.
CHECK (valid range)	current_horde, horde_number, level_number	Values must stay within valid game-design ranges (e.g., level_number between 1 and 3).
UNIQUE	CARD_UPGRADE (player_id, card_id)	Each player may have only one upgrade record per card. Prevents duplicate upgrade entries.
UNIQUE	RUN_HORDE (run_id, horde_id)	The same horde cannot be recorded twice within the same run. Prevents duplicated progress entries.
UNIQUE	HORDE (level_id, horde_number)	Within a level, horde numbers must be unique. Prevents two hordes sharing the same position.
CHECK (valid result)	RUN.result	Only accepts: 'victory', 'defeat', or 'in_progress'. Prevents arbitrary or invalid strings.
CHECK (date order)	RUN (started_at, ended_at)	ended_at must be NULL or greater than started_at. A run cannot end before it begins.

These constraints should be enforced at the database level (DDL) and not solely in application logic, ensuring data integrity regardless of which system accesses the database.

This schema is an initial version. It has not been normalized to a specific normal form, as the design may change as game development progresses.

UML

